

# **Documentación Técnica**

## **Kart AI Racing**

*Proyecto Final — Máster de Inteligencia Artificial y Big Data*

Pol Panyella  
Ronald Intriago  
JiaJiao Xu  
Jordi Vidal

|                                                        |           |
|--------------------------------------------------------|-----------|
| <b>1. Introducción</b>                                 | <b>3</b>  |
| <b>2. Objetivos del proyecto</b>                       | <b>4</b>  |
| <b>3. Requisitos funcionales</b>                       | <b>5</b>  |
| 3.1 Requisitos del entorno de videojuego               | 5         |
| 3.2 Requisitos de aprendizaje por refuerzo             | 6         |
| 3.3 Requisitos de los sistemas de clasificación        | 6         |
| <b>4. Requisitos no funcionales</b>                    | <b>8</b>  |
| 4.1 Rendimiento                                        | 8         |
| 4.2 Mantenibilidad y reproducibilidad                  | 8         |
| 4.3 Calidad y evaluación de modelos                    | 8         |
| 4.4 Compatibilidad y usabilidad                        | 9         |
| <b>5. Arquitectura del sistema</b>                     | <b>10</b> |
| 5.1 Arquitectura del juego                             | 10        |
| 5.1.1 Flujo principal de ejecución de una carrera      | 11        |
| 5.2 Herramientas de diagnóstico in-game                | 11        |
| <b>6. Aprendizaje por refuerzo con Unity ML-Agents</b> | <b>12</b> |
| 6.1 Algoritmo: PPO (Proximal Policy Optimization)      | 12        |
| 6.2 Agente de conducción - KartAgent                   | 12        |
| 6.2.1 Espacio de observaciones                         | 13        |
| 6.2.2 Espacio de acciones                              | 14        |
| 6.2.3 Sensor de Rayos personalizado                    | 15        |
| 6.2.4 Función de recompensa                            | 15        |
| 6.2.5 Fin de episodio                                  | 16        |
| 6.2.6 Hiperparámetros de Entrenamiento                 | 16        |
| 6.3 Proceso de Entrenamiento Iterativo                 | 17        |
| 6.4 Resultados: Métricas de TensorBoard                | 18        |
| <b>7. Sistema de decisión de power-ups</b>             | <b>23</b> |
| 7.1 Descripción general                                | 23        |
| 7.2 Formulación del problema                           | 23        |
| 7.2.1 ¿Por qué no usar reglas fijas?                   | 23        |
| 7.2.2 Estructura de dataset de entreno                 | 24        |
| 7.3 Dataset sintético y EDA                            | 24        |
| 7.3.1 Creación del dataset semilla                     | 24        |
| 7.3.2 EDA del dataset semilla                          | 25        |
| 7.4 Dataset sintético y Data Augmentation              | 28        |
| 7.4.1 Motivación y enfoque                             | 28        |
| 7.4.2 Mecanismo de generación                          | 28        |
| 7.4.3 Dataset de producción                            | 28        |
| 7.4.4 Limitaciones de los datos sintéticos             | 30        |

|                                                      |           |
|------------------------------------------------------|-----------|
| 7.5 Entrenamiento del Modelo                         | 30        |
| 7.5.1 Uso de Random Forest                           | 30        |
| 7.5.2 Hiperparámetros                                | 30        |
| 7.5.3 Pipeline de entrenamiento                      | 31        |
| 7.6 Resultados de Evaluación                         | 31        |
| 7.6.1 Métricas globales                              | 31        |
| 7.6.2 Classification report por clase                | 31        |
| 7.6.3 Inferencia en el runtime (C#)                  | 32        |
| 7.8 Evaluación In-Game                               | 33        |
| 7.8.1 Distribuciones de power-ups observada          | 33        |
| <b>8. Clasificador de comportamiento del jugador</b> | <b>34</b> |
| <b>8.1 Arquetipos de Jugador</b>                     | <b>34</b> |
| <b>8.2 Recolección de datos</b>                      | <b>35</b> |
| <b>8.2.1 PlayerLapDataRecorder</b>                   | <b>35</b> |
| <b>8.3 Datos Sintéticos</b>                          | <b>36</b> |
| <b>8.3.1 Limitaciones</b>                            | <b>36</b> |
| <b>8.3.2 Diseño del Dataset Sintético</b>            | <b>36</b> |
| <b>8.3.3 Restricciones de Realismo Aplicadas</b>     | <b>37</b> |
| <b>8.3.4 Limitaciones de los Datos Sintéticos</b>    | <b>37</b> |
| <b>8.3.5 Feature Engineering</b>                     | <b>38</b> |
| <b>8.4 Entreno y comparación de modelos</b>          | <b>38</b> |
| <b>8.4.1 División de los Datos</b>                   | <b>38</b> |
| <b>8.4.2 Gestión del Desequilibrio de Clases</b>     | <b>39</b> |
| <b>8.4.3 Métrica de Evaluación</b>                   | <b>39</b> |
| <b>8.4.4 Modelos Evaluados</b>                       | <b>39</b> |
| <b>8.4.5 Resultados en el Conjunto de Validación</b> | <b>40</b> |
| <b>8.4.6 Validación Cruzada</b>                      | <b>40</b> |
| <b>9. Limitaciones y trabajo futuro</b>              | <b>41</b> |
| <b>9.1 Limitaciones actuales</b>                     | <b>41</b> |
| <b>9.1.1 En el bloque de IA</b>                      | <b>41</b> |
| <b>9.1.2 En el entorno de videojuego</b>             | <b>41</b> |
| <b>9.2 Trabajo futuro</b>                            | <b>42</b> |
| <b>10. Conclusiones</b>                              | <b>43</b> |
| <b>11. Referencias</b>                               | <b>44</b> |

# 1. Introducción

Kart AI Racing es un videojuego de carreras arcade desarrollado en Unity 6 como proyecto final del Máster de Inteligencia Artificial y Big Data. El videojuego es el entorno de pruebas; el contenido central del proyecto son los sistemas de inteligencia artificial y el tratamiento de datos que se ejecutan dentro de él.

El proyecto integra aprendizaje por refuerzo (entrenamiento de agentes de conducción y de uso de power-ups con Unity ML-Agents y el algoritmo PPO) y aprendizaje supervisado clásico (un sistema de decisión de power-ups basado en Random Forest y otro sistema de clasificación para categorizar al jugador).

El videojuego en sí (conducción, checkpoints, vueltas, posiciones, UI) se ha desarrollado como base funcional sobre la que se muestran los sistemas de IA.

## 2. Objetivos del proyecto

Nuestra meta es desarrollar un videojuego que contenga bots con un comportamiento semejante al de un jugador, además de integrar algoritmos de ML para tener una visión de lo que pasa en la partida y lo que hace el jugador.

Ya que el proyecto se compone de dos bloques de trabajo bien diferenciados, diferenciar objetivos del proyecto en bloques:

1. **Bloque de IA y datos:** Modelo de aprendizaje por refuerzo para los bots, así como la recolección de datos de partida para modelos de clasificación que ayuden a los bots a escoger power-up y a clasificar tipos de jugadores.
2. **Bloque de videojuego** prototipo arcade jugable en Unity 6 con sistema de carrera completo. Sirve como entorno donde los sistemas de IA pueden ejecutarse y demostrarse.

## 3. Requisitos funcionales

### 3.1 Requisitos del entorno de videojuego

#### **RF-01. Inicio de carrera**

El sistema debe permitir iniciar una carrera desde una escena de Unity.

#### **RF-02. Control del jugador**

El jugador controla un kart mediante teclado (acelerar, frenar, girar, usar power-up, resetear).

#### **RF-03. Conducción arcade**

El kart utiliza físicas Rigidbody con aceleración, frenado, giro, límite de velocidad y reducción de derrape.

#### **RF-04. Cámara y reset**

Cámara de seguimiento del kart del jugador y posibilidad de reset si queda bloqueado.

#### **RF-05. Cuenta atrás y estados**

Estados de la carrera: preparación, cuenta atrás, carrera activa y finalizada.

#### **RF-06. Registro y colocación**

Registro de corredores (jugador y bots) y colocación en parrilla de salida.

#### **RF-07. Vueltas y final**

Gestión del número de vueltas completadas y condición de fin de carrera.

#### **RF-08. Sistema de checkpoints**

Circuito dividido en checkpoints que validan el progreso de cada corredor, comprobando el orden.

#### **RF-09. Cálculo de progreso**

Cada corredor almacena su vuelta actual, último checkpoint alcanzado y distancia al siguiente.

#### **RF-10. Posiciones en tiempo real**

El sistema calcula la posición de cada corredor combinando vueltas, checkpoints y distancia al siguiente checkpoint.

#### **RF-11. Bots funcionales**

Bots que siguen el circuito usando el sistema de checkpoints, con recuperación automática si se bloquean.

#### **RF-12. UI básica**

Menú principal, HUD con tiempo/vuelta/posición, pantalla de resultados, modo espectador y pausa.

## 3.2 Requisitos de aprendizaje por refuerzo

### RF-13. Escenas de entrenamiento

Escenas dedicadas para entrenamiento de KartAgent (conducción) y PowerUpAgent (decisión de power-ups), separadas de la escena de carrera para no impactar el rendimiento.

### RF-14. KartAgent

Agente que recibe observaciones del entorno y produce acciones de aceleración, frenado y giro mediante una política PPO.

### RF-15. Observaciones de KartAgent

Velocidad, dirección al siguiente checkpoint, distancia al objetivo, progreso de checkpoints, estado en suelo, orientación del kart y sensores de raycast.

### RF-16. Función de recompensa

Recompensas positivas por avanzar y pasar checkpoints; penalizaciones por chocar, quedarse parado, ir en sentido contrario o salirse de la pista.

### RF-17. Configuración PPO

Hiperparámetros definidos en YAML (batch\_size, buffer\_size, learning\_rate, beta, epsilon, lambda, num\_epoch, hidden\_units, num\_layers, gamma).

### RF-18. PowerUpAgent

Agente PPO que decide qué power-up usar en función del contexto de carrera (variante de aprendizaje por refuerzo del problema que también resuelve el Random Forest).

### RF-19. Evidencias de entrenamiento

TensorBoard logs de cada run, modelos exportados y configuración versionada para reproducibilidad.

## 3.3 Requisitos de los sistemas de clasificación

### RF-20. Dataset semilla

Datasets iniciales con datos suficientes para su explotación y entreno.

### RF-21. Features

El contexto de carrera se modela con 5 features: recta (booleano), enemigos\_delante, enemigos\_atras, platanos\_delante, conchas\_atras (todos enteros 0–5).

### RF-22. Clases objetivo

El sistema clasifica entre 4 power-ups: banana, shell, mushroom, star.

### RF-23. EDA reproducible

Script de EDA que genera resumen de filas, duplicados, balance de clases, distribución por feature y medias por clase, exportado a Markdown.

### RF-24. Data augmentation

Script que genera datasets balanceados o sesgados (perfiles balanced/agresivo/pacífico) con tamaño configurable y semilla reproducible.

### RF-25. Entrenamiento

Script que entrena los modelos y los guarda.

### RF-26. Métricas de evaluación

Después de cada entrenamiento se generan accuracy, classification report (precision/recall/f1 por clase) y matriz de confusión.

#### **RF-27. Exportación a JSON**

El modelo entrenado se serializa a JSON con la estructura completa de los árboles, consumible desde C# en Unity.

#### **RF-28. Inferencia en Unity**

Unity carga el JSON al inicio y ejecuta la predicción de los modelos de clasificación, cuando sea necesario, en runtime, sin requerir Python.

#### **RF-29. Recolección de datos reales**

El sistema registra las decisiones de power-up del jugador durante partidas y guarda los logs por lap para uso posterior.



## 4. Requisitos no funcionales

### 4.1 Rendimiento

#### **RNF-01. Fluidez del juego**

El prototipo debe ejecutarse de forma fluida sin caídas graves de rendimiento durante la carrera.

#### **RNF-02. Inferencia rápida**

La inferencia del Random Forest en Unity debe ejecutarse en <1ms por decisión para no afectar el frame rate.

#### **RNF-03. Entrenamiento separado**

Las escenas de entrenamiento de ML-Agents son independientes de la escena jugable para evitar sobrecarga.

### 4.2 Mantenibilidad y reproducibilidad

#### **RNF-04. Código modular**

Código separado por sistemas (Core, Kart, AI, UI, PowerUps) en C#, y scripts de datos en Python.

#### **RNF-05. Pipeline reproducible**

Todo el pipeline de datos (semilla → augmentation → entrenamiento → exportación) se ejecuta con scripts versionados y semillas fijas.

#### **RNF-06. Versionado de modelos**

Los modelos exportados (joblib + JSON + ONNX) se versionan junto al código para garantizar reproducibilidad.

#### **RNF-07. Configuración explícita**

Los hiperparámetros de ML-Agents se definen en archivos YAML versionados.

### 4.3 Calidad y evaluación de modelos

#### **RNF-08. Accuracy mínima**

El modelo Random Forest debe alcanzar al menos un 90% de accuracy en el test set para considerarse válido.

#### **RNF-09. Matriz de confusión**

Cada modelo debe acompañarse de su matriz de confusión sobre el test set.

#### **RNF-10. Convergencia RL**

La curva de reward del KartAgent debe mostrar convergencia visible y mantenerse estable durante al menos 100k steps.

#### **RNF-11. Class weight**

Los modelos supervisados utilizan class\_weight=balanced para mitigar desbalances residuales.

## 4.4 Compatibilidad y usabilidad

### **RNF-12. Compatibilidad Unity 6**

El proyecto se desarrolla y prueba en Unity 6 con URP, Input System y UGUI.

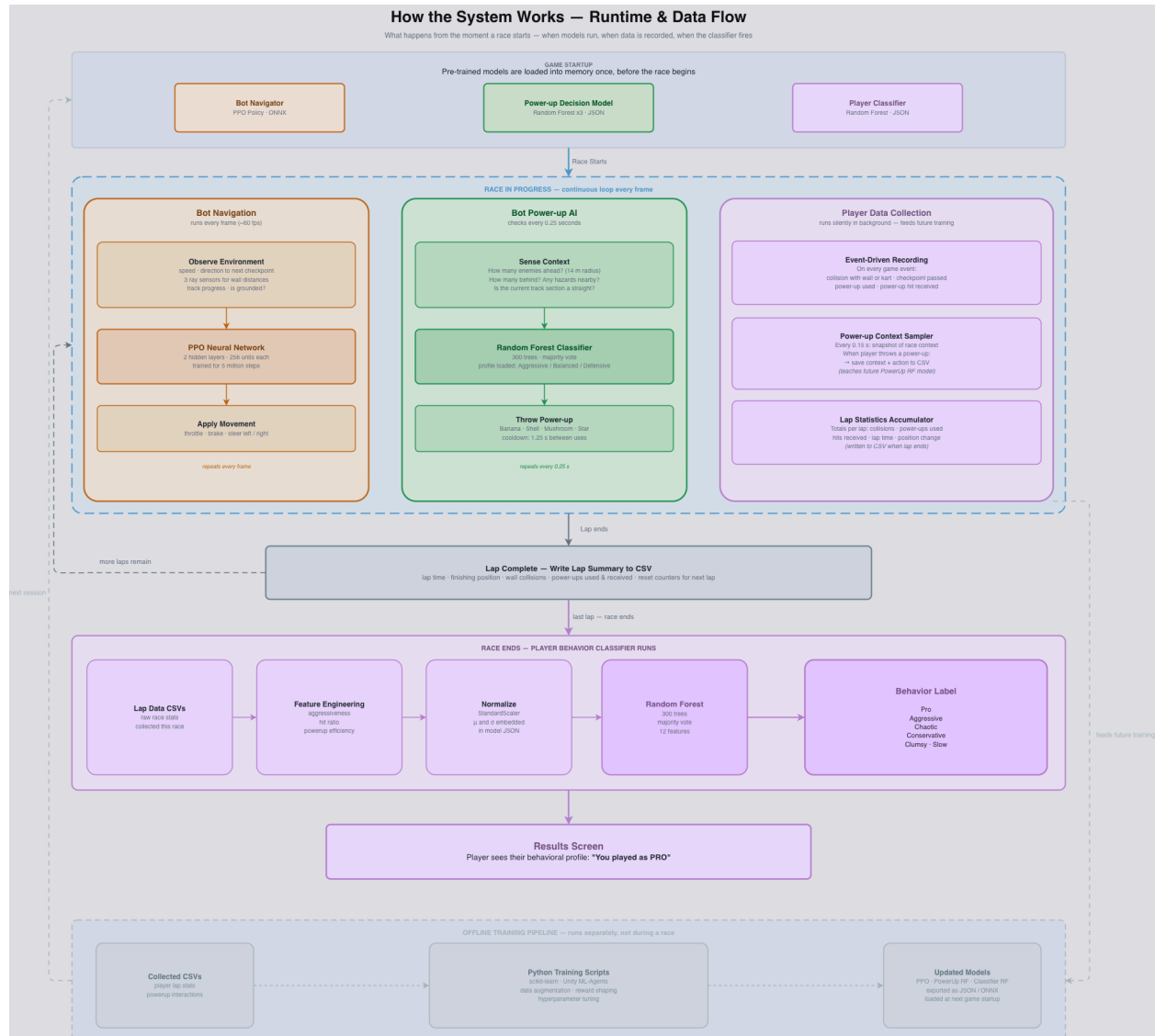
### **RNF-13. Controles sencillos**

El jugador controla el kart con WASD/flechas, espacio, R.

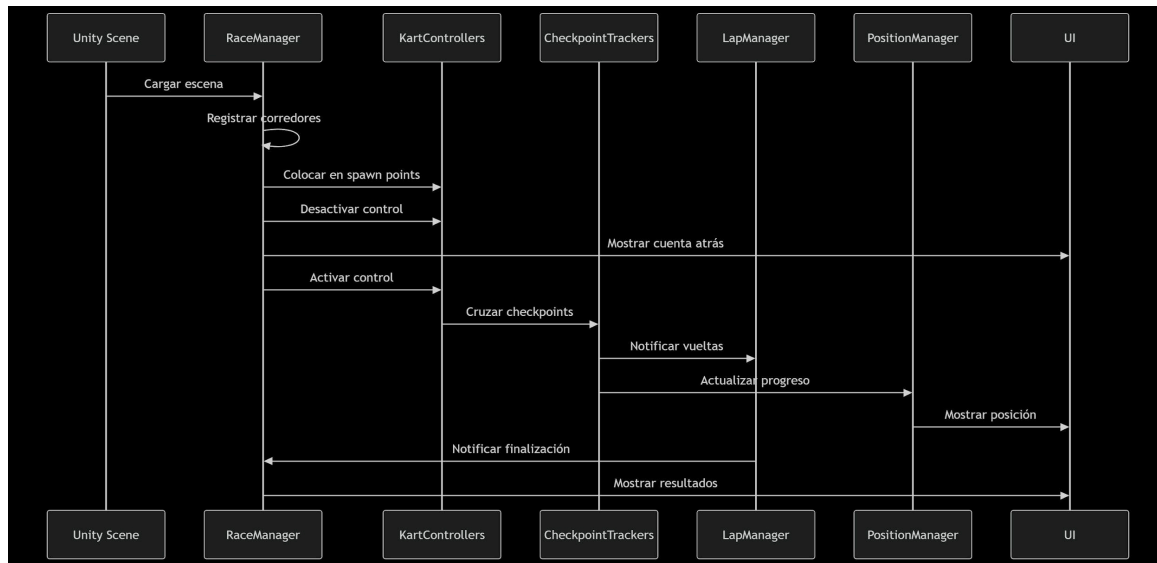
## 5. Arquitectura del sistema

La arquitectura separa el entorno de videojuego (Unity, C#) del bloque de IA y datos (Python para entrenamiento, JSON/ONNX para despliegue, C# para inferencia). Esta separación permite que el pipeline de datos sea reproducible fuera del juego, y que Unity sólo necesite cargar artefactos ya entrenados.

### 5.1 Arquitectura del juego



### 5.1.1 Flujo principal de ejecución de una carrera



Los sistemas de IA se conectan a esta arquitectura en tres puntos concretos:

- **KartAgent** sustituye o complementa a AIKartInput como controlador de bots, alimentando directamente a KartController con acciones aprendidas.
- **Random Forest de power-ups** se invoca desde PowerUpManager cuando un corredor recoge una caja, recibiendo el contexto del corredor (PositionManager + estado del circuito) y devolviendo qué power-up asignar.
- **Clasificador de comportamiento del jugador** se ejecuta una vez al final de la carrera, recibiendo los datos del jugador y devolviendo una etiqueta con el resultado de su clasificación.

## 5.2 Herramientas de diagnóstico in-game

Un elemento característico de la arquitectura es el uso de herramientas de diagnóstico integradas en Unity que generan reportes de texto plano durante o después de una sesión de juego. Estos reportes son la evidencia empírica del funcionamiento del sistema integrado.

| Herramienta                  | Salida                                                                                                   | Carpeta de destino                 |
|------------------------------|----------------------------------------------------------------------------------------------------------|------------------------------------|
| Export Training Scene Report | Diagnóstico de configuración de la escena de entrenamiento ML-Agents (referencias, prefabs, componentes) | Assets/Documentation/Diagnostics / |
| Export Power-Up Usage Report | Métricas de uso de power-ups por kart, Decision Accuracy, Execution Success Rate, Learning Verdict       | Assets/Informes power ups/         |

## 6. Aprendizaje por refuerzo con Unity ML-Agents

### 6.1 Algoritmo: PPO (Proximal Policy Optimization)

El motor de juego Unity 6, incluye ML-Agents. Es una toolkit que permite a Unity usar entornos para entrenar modelos mediante reinforcement learning e imitation learning.

Se ha elegido PPO como algoritmo de aprendizaje por refuerzo. PPO es un algoritmo on-policy basado en gradiente de política con clipping, ampliamente usado en problemas de control continuo y discreto. ML-Agents lo incluye como trainer por defecto y proporciona implementación estable en PyTorch.

Las razones de elegir PPO en este proyecto son:

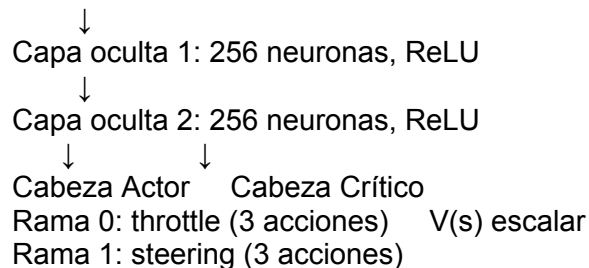
- Estabilidad del entrenamiento: el clipping evita actualizaciones agresivas de la política, lo que reduce oscilaciones en el reward.
- Compatibilidad nativa con ML-Agents: el trainer está optimizado e integrado con TensorBoard.
- Es el algoritmo más documentado dentro del ecosistema Unity ML-Agents, lo que facilita la reproducibilidad y depuración.

### 6.2 Agente de conducción - KartAgent

**KartAgent** hereda de la clase **Agent** de **ML-Agents** y gestiona el ciclo completo de entrenamiento del kart: inicialización de episodio, recogida de observaciones, traducción de acciones a entradas de control y cálculo de recompensas.

La red usa una arquitectura **actor-crítico con parámetros compartidos**: las capas ocultas son comunes al actor (que decide acciones) y al crítico (que estima el valor del estado). La configuración es:

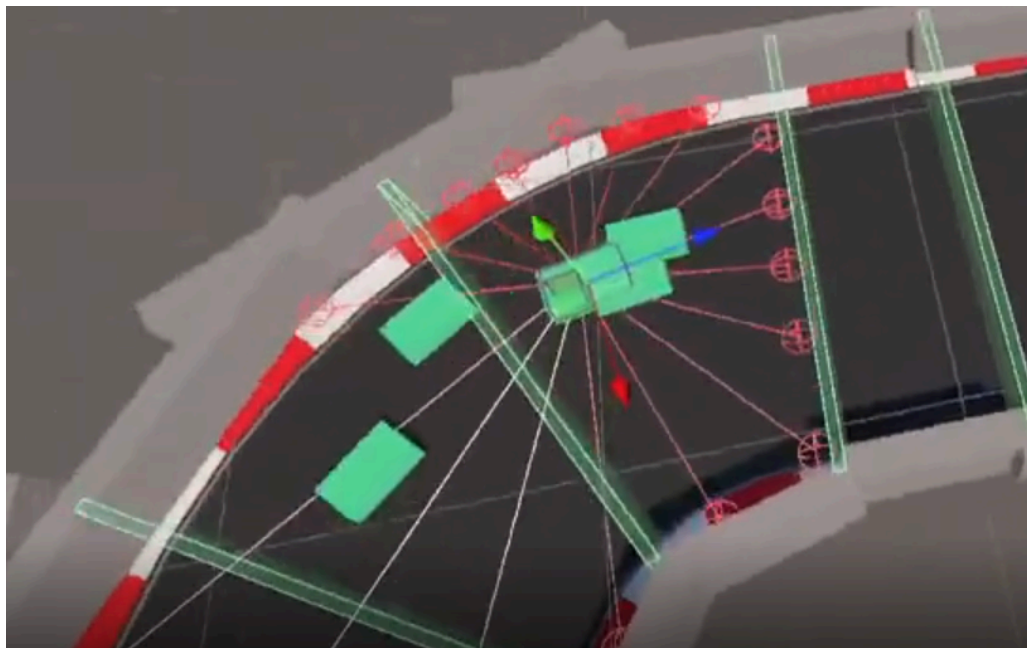
Entrada: 13 observaciones vectoriales + observaciones del sensor de rayos



### 6.2.1 Espacio de observaciones

El KartAgent recibe en cada step (época) un vector de observaciones que describe su situación en el circuito. Las observaciones se han diseñado para que el agente pueda inferir, sin información global del mapa, hacia dónde debe ir y cómo está orientado:

| #  | Variable                                  | Implementación                                            | Rango                |
|----|-------------------------------------------|-----------------------------------------------------------|----------------------|
| 1  | Velocidad lateral local                   | <code>localVelocity.x / maxSpeed</code>                   | <code>[-1, 1]</code> |
| 2  | Velocidad frontal local                   | <code>localVelocity.z / maxSpeed</code>                   | <code>[-1, 1]</code> |
| 3  | Velocidad escalar normalizada             | <code>GetCurrentSpeed() / maxSpeed</code>                 | <code>[0, 2]</code>  |
| 4  | Dirección al checkpoint (X local)         | <code>planarDirection.x</code> — vector unitario plano XZ | <code>[-1, 1]</code> |
| 5  | Dirección al checkpoint (Z local)         | <code>planarDirection.y</code> — vector unitario plano XZ | <code>[-1, 1]</code> |
| 6  | Distancia al checkpoint normalizada       | <code>planarTarget.magnitude / 25f</code>                 | <code>[0, 4]</code>  |
| 7  | Alineación con el forward del checkpoint  | <code>Dot(transform.forward, checkpoint.forward)</code>   | <code>[-1, 1]</code> |
| 8  | Alineación con la dirección al checkpoint | <code>Dot(transform.forward, dirToCheckpoint)</code>      | <code>[-1, 1]</code> |
| 9  | Contacto con el suelo                     | <code>kartController.IsGrounded</code>                    | <code>{0, 1}</code>  |
| 10 | Estado fuera de pista                     | <code>_isOffTrack</code>                                  | <code>{0, 1}</code>  |
| 11 | Progreso de checkpoints                   | <code>lastPassedIndex / checkpointCount</code>            | <code>[0, 1]</code>  |
| 12 | Progreso de vueltas                       | <code>completedLaps / lapsToWin</code>                    | <code>[0, 1]</code>  |
| 13 | Verticalidad                              | <code>transform.up.y</code>                               | <code>[-1, 1]</code> |



En esta imagen se observa la *grid* de los elementos del espacio de observación.

### 6.2.2 Espacio de acciones

El agente produce acciones que se mapean a los controles del kart equivalentes a los del jugador. Las acciones se procesan internamente como combinaciones de aceleración, frenado y giro, alimentando directamente a KartController.

| Rama         | Índice | Acción                                                                                 |
|--------------|--------|----------------------------------------------------------------------------------------|
| 0 — Throttle | 0      | Sin entrada                                                                            |
| 0 — Throttle | 1      | <code>acceleration = 1f</code>                                                         |
| 0 — Throttle | 2      | Si velocidad > 1.5: <code>brake = 1f</code> ; si no: <code>acceleration = -0.5f</code> |
| 1 — Steering | 0      | Sin entrada                                                                            |
| 1 — Steering | 1      | <code>steering = -1f</code> (izquierda)                                                |
| 1 — Steering | 2      | <code>steering = 1f</code> (derecha)                                                   |

La acción throttle=2 aplica frenado diferenciado según velocidad: a alta velocidad frena, a baja velocidad activa marcha atrás.

### 6.2.3 Sensor de Rayos personalizado

El agente usa **CheckpointAwareRayPerceptionSensorComponent3D**, una extensión del sensor estándar de ML-Agents desarrollada específicamente para este proyecto. El sensor estándar detecta todos los checkpoints de la pista, lo que confunde al agente al percibir checkpoints ya superados como si fueran objetivos. El sensor personalizado añade dos filtros:

- Ignora checkpoints ya superados (**ignorePassedCheckpoints**): descarta el último checkpoint pasado.
- Ventana de detección limitada (**limitCheckpointDetectionWindow**): solo muestra los próximos **additionalVisibleCheckpointsAhead** = 3 checkpoints por delante.

El sensor lanza  $2 \times \text{raysPerDirection} + 1 = 7$  rayos con apertura de  $70^\circ$ , longitud de 20 unidades y radio de esfera de 0.5. Internamente usa **Physics.SphereCastAll**, ordena los impactos por distancia y aplica los filtros antes de reportar el primer hit válido.

### 6.2.4 Función de recompensa

Definidas en **AgentRewardManager.cs**. La configuración final concentra la señal de aprendizaje en eventos discretos claros:

| Evento                           | Valor            |
|----------------------------------|------------------|
| Pasar checkpoint correcto        | +1.0             |
| Recoger moneda                   | +0.05 × cantidad |
| Recibir impacto de banana        | -0.12            |
| Recibir impacto de shell         | -0.08            |
| Caída fuera de límites de altura | -1.0             |

**Configuradas a 0 por defecto (ajustables desde el Inspector):**

Penalizaciones como alejarse del checkpoint (-0.02), idle (-0.01/s), dirección incorrecta (-0.0125/s), colisión con muro (-0.035), contacto continuo con muro (-0.02/s), entrar en zona off-track (-0.05) y penalización por paso (-0.005) están definidas en el código pero desactivadas en la configuración final. La observación experimental durante el proceso de entrenamiento iterativo fue que la señal de checkpoint (+1.0) combinada con el reinicio por caída es suficiente para guiar el aprendizaje sin introducir gradientes contradictorios.



## 6.2.5 Fin de episodio

El episodio termina cuando:

- El kart completa una vuelta (`endEpisodeOnLapCompletion = true`)
- El kart cae por debajo de la altura `outOfBoundsHeight = -2f`
- El kart entra en una zona off-track (`endEpisodeOnOffTrack = true`)
- El entrenador interrumpe el episodio al alcanzar `max_steps`

## 6.2.6 Hiperparámetros de Entrenamiento

La configuración del entrenamiento se almacena en un archivo yaml, en nuestro caso se encuentra en la ruta `Assets/MLAgents/Config/kart_agent_config.yaml`.

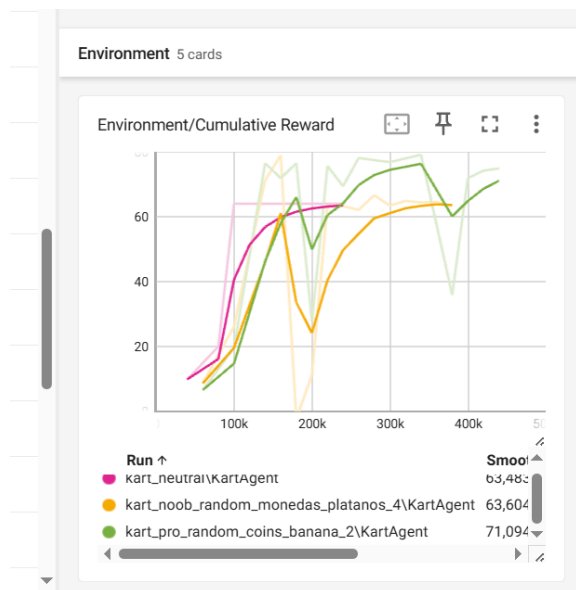
| Parámetro                           | Valor               | Efecto                                                                     |
|-------------------------------------|---------------------|----------------------------------------------------------------------------|
| <code>batch_size</code>             | 254                 | Muestras por minilote                                                      |
| <code>buffer_size</code>            | 10240               | Experiencias recogidas antes de cada ciclo de entrenamiento                |
| <code>learning_rate</code>          | 3e-4                | Tasa de aprendizaje inicial (Adam)                                         |
| <code>learning_rate_schedule</code> | <code>linear</code> | Decae linealmente hasta 0 al finalizar                                     |
| <code>beta</code>                   | 5e-3                | Peso del bonus de entropía                                                 |
| <code>epsilon</code>                | 0.2                 | Límite de clipping de PPO: la política no puede cambiar más del $\pm 20\%$ |
| <code>lambd</code>                  | 0.95                | Parámetro $\lambda$ de GAE                                                 |
| <code>num_epoch</code>              | 3                   | Épocas de entrenamiento por ciclo                                          |
| <code>gamma</code>                  | 0.99                | Factor de descuento                                                        |
| <code>hidden_units</code>           | 256                 | Neuronas por capa oculta                                                   |
| <code>num_layers</code>             | 2                   | Capas ocultas                                                              |
| <code>max_steps</code>              | 5 000 000           | Pasos totales de entrenamiento                                             |
| <code>time_horizon</code>           | 64                  | Longitud del segmento GAE                                                  |
| <code>summary_freq</code>           | 20 000              | Frecuencia de escritura a TensorBoard                                      |

## 6.3 Proceso de Entrenamiento Iterativo

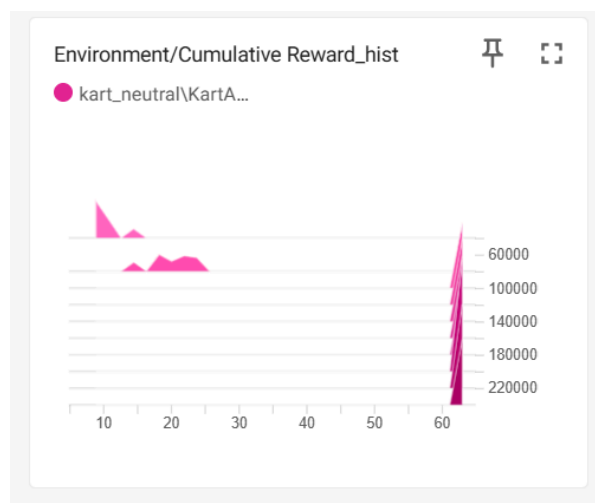
Los logs se almacenan en `Assets/MLAgents/TrainingLogs/`, con más de 40 runs que reflejan un proceso experimental de ajuste de recompensas y configuración:

| Fase                    | Runs representativos                                                                       | Descripción                                     |
|-------------------------|--------------------------------------------------------------------------------------------|-------------------------------------------------|
| Exploración inicial     | <code>kart_02</code> , <code>kart_04</code> – <code>kart_08</code>                         | Pruebas de configuración base                   |
| Perfil básico           | <code>kart_noob</code> , <code>kart_noob2</code>                                           | Comportamiento básico de seguimiento            |
| Con objetos del entorno | <code>kart_noob_random_monedas_platanos_1-4</code>                                         | Incorporación de monedas y bananas              |
| Ajuste intermedio       | <code>kart_neutral</code> ,<br><code>kart_neutral_random_monedas_platanos</code>           | Refinamiento de recompensas                     |
| Pista 360°              | <code>kart360_1</code> – <code>kart360_4</code> ,<br><code>kart_360_checkpoint_back</code> | Circuito completo, checkpoint tracking mejorado |
| Comportamiento avanzado | <code>kart_pro</code> – <code>kart_pro5</code> ,<br><code>kart_pro_doble_sensor</code>     | Doble sensor, mayor complejidad                 |
| Circuito final          | <code>kart_pista_completa</code> ,<br><code>kart_pista_completa_recompensa_simple</code>   | Entrenamiento sobre la pista definitiva         |

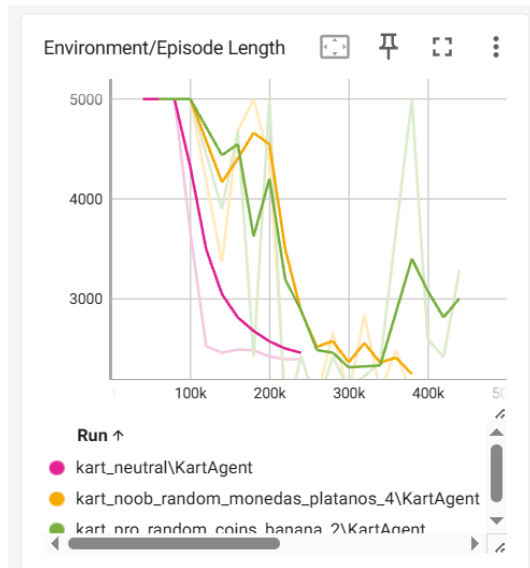
## 6.4 Resultados: Métricas de TensorBoard



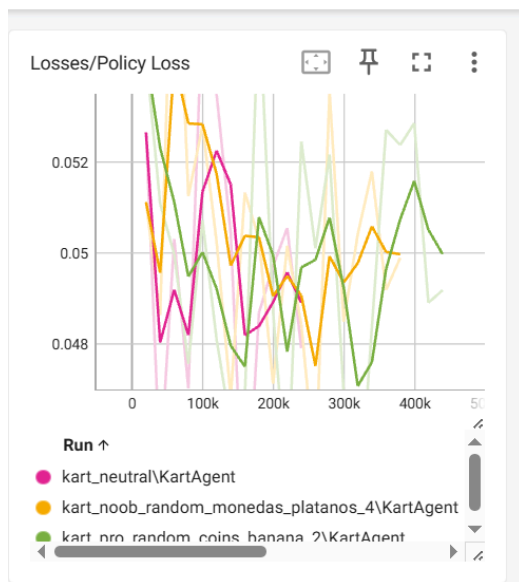
Todos los modelos entrenados convergen a valores finales de **60–70 puntos** de recompensa acumulada por episodio. Los picos descendentes ocasionales son normales: responden a momentos de exploración, reinicios de la comunicación Unity–Python, o actualizaciones de política propias de PPO que pueden empeorar transitoriamente antes de mejorar.



El histograma muestra que al final del entrenamiento la masa de episodios se concentra en el rango **60–80**, no solo la media. Esto indica que la mayoría de episodios terminan bien, no que unos pocos episodios buenos eleven artificialmente el promedio. Es una señal más robusta de aprendizaje genuino.

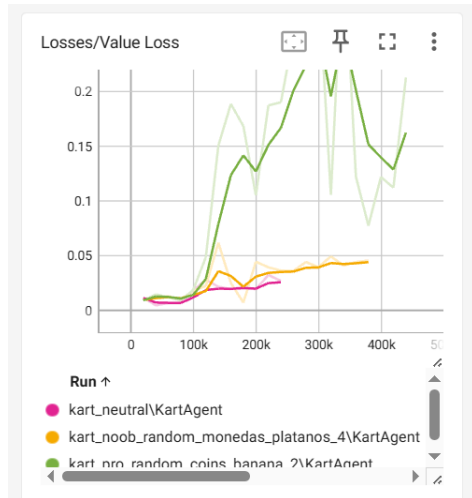


Partió de aproximadamente **5000 pasos** y descendió a **2200–3000**. La reducción de longitud junto con el aumento de recompensa confirma el aprendizaje: los agentes completan el circuito en menos pasos porque han aprendido trayectorias más eficientes. Si la longitud hubiera bajado con la recompensa también bajando, indicaría fallos tempranos en lugar de eficiencia.

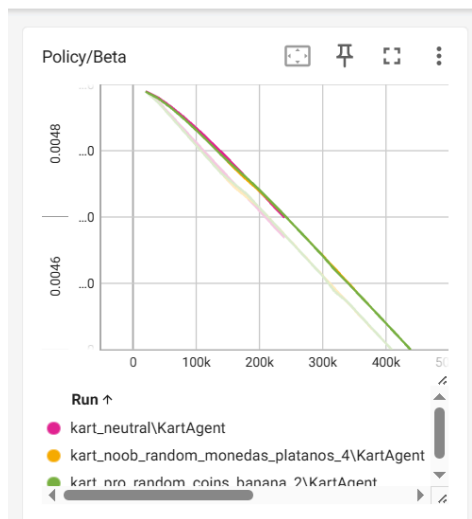


Se mantuvo estable en torno a **0.048–0.052** a lo largo del entrenamiento. En PPO esta pérdida no se interpreta como en clasificación supervisada (donde menor siempre es mejor): oscila por diseño porque el clipping la regula activamente.

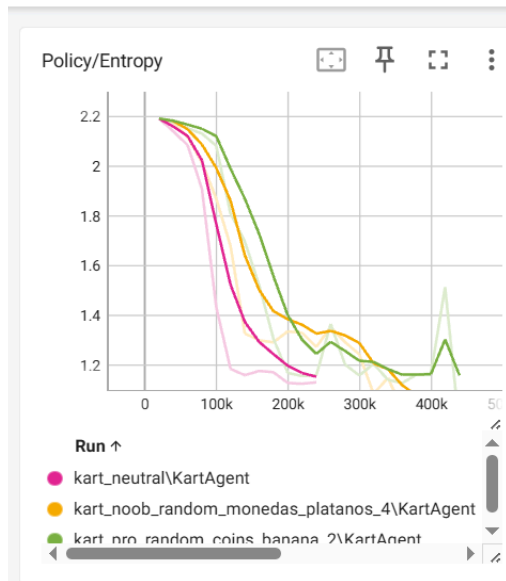
Valores compactos sin tendencia creciente confirman que el parámetro **epsilon: 0.2** funcionó correctamente.



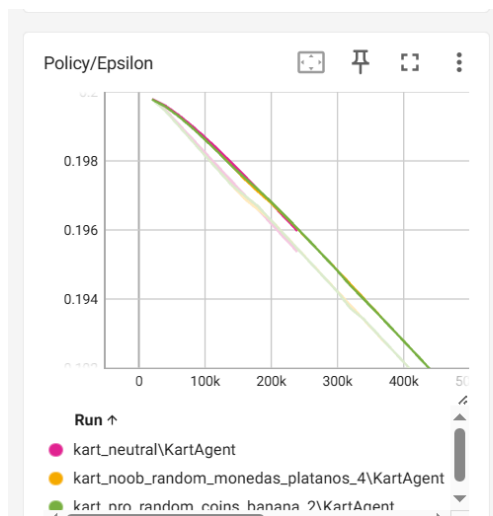
La mayoría de los modelos convergieron a valores entre **0.02 y 0.04**. Un modelo presentó un valor más alto ( $\sim 0.16$ ), consecuencia de una distribución de recompensas más variable en su configuración de entrenamiento; aun así alcanzó alta recompensa acumulada, por lo que no representó un problema de aprendizaje real.



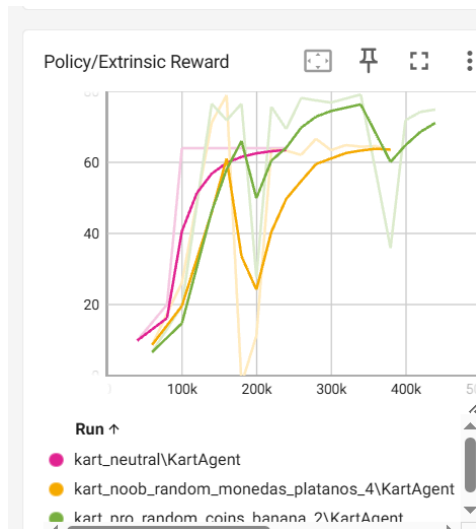
Decrece linealmente desde su valor inicial hasta 0, coherente con **beta\_schedule: linear**. La caída suave indica que el agente exploró ampliamente al inicio y fue comprometiendo su comportamiento a medida que avanzó el entrenamiento. Una caída brusca prematura hubiera indicado pérdida de exploración antes de encontrar la política óptima.



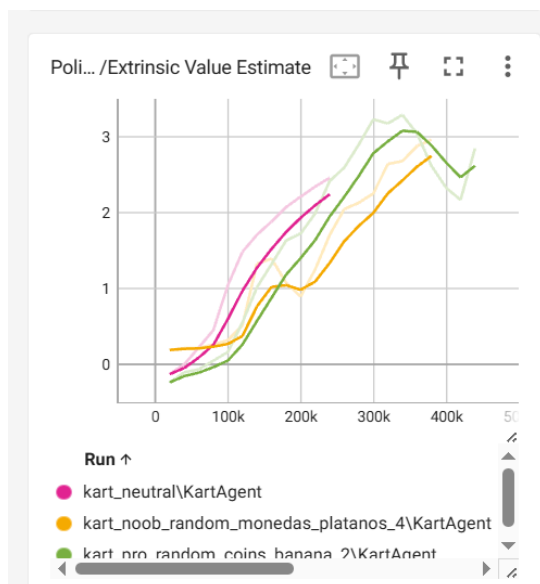
Partió del máximo teórico para 9 acciones posibles ( $\sim 2.2$ ) y descendió a **1.1–1.3** al finalizar. Esta evolución confirma que los agentes desarrollaron una política definida sin colapsar a un único comportamiento repetitivo. Un descenso a valores cercanos a 0 de forma prematura hubiera indicado convergencia a un óptimo local por falta de exploración.



Decrece lentamente de 0.2 hacia **0.193–0.196**, coherente con **epsilon\_schedule: linear**. No es una métrica de rendimiento; es el parámetro de control que limita el cambio de política por actualización, haciéndose marginalmente más conservador en las fases finales.



Prácticamente idéntica a **Cumulative Reward** porque el proyecto no usa señales de recompensa intrínseca en el entrenamiento PPO puro. Los valores finales de 60–75 reflejan directamente el rendimiento del agente en el circuito.



Ascendió desde valores cercanos a 0 hasta **2.5–3**, acompañando el crecimiento de la recompensa real de forma proporcional. Que el crítico estime valores crecientes en paralelo con la recompensa real indica que aprendió correctamente a identificar estados prometedores. Si este valor hubiera crecido desproporcionadamente mientras la recompensa real se mantenía plana, indicaría sobreestimación del valor; ese comportamiento no se observó.

## 7. Sistema de decisión de power-ups

### 7.1 Descripción general

Este capítulo describe el pipeline completo para entrenar un bot capaz de decidir qué power-up usar en cada momento de la partida a partir del contexto de carrera observable en ese instante. El modelo se entrena en Python, se exporta como estructura JSON y se ejecuta en tiempo real dentro de Unity sin ninguna dependencia de Python en runtime.

En el juego final, los tres bots del juego (Kart\_Pro, Kart\_Noob, Kart\_Neutral) utilizan el modelo **powerups\_random\_forest.json**, entrenado sobre el dataset balanceado. El sistema fue diseñado para soportar personalidades diferenciadas mediante modelos distintos, y los modelos alternativos fueron entrenados y validados, pero la integración de perfiles por bot queda como trabajo futuro por temas de plazos de tiempo.

### 7.2 Formulación del problema

El problema se plantea como una clasificación supervisada de 4 clases: dado el contexto actual de un corredor en un instante determinado, predecir qué power-up debería usar. Las clases objetivo son las siguientes:

| Power-Up | Descripción                                 |
|----------|---------------------------------------------|
| Banana   | Ataque a rivales que están detrás.          |
| Shell    | Ataque a rivales que estén delante.         |
| Mushroom | Turbo de velocidad. Ótimo en tramos rectos. |
| Star     | Invencibilidad temporal                     |

Cuando el power-up predicho en primer lugar no está disponible en inventario, el modelo intentará automáticamente escoger el siguiente candidato por orden de votos hasta encontrar uno utilizable.

#### 7.2.1 ¿Por qué no usar reglas fijas?

La alternativa más directa habría sido un árbol de condicionales codificado a mano. El proyecto incluye exactamente eso en KartPowerUpBotBrain, dejado como sistema de referencia:

```
si hazards cercanos  $\geq$  umbral  $\rightarrow$  star  
si enemigos delante  $> 0$   $\rightarrow$  shell  
si enemigos detrás  $> 0$   $\rightarrow$  banana  
si recta limpia  $\rightarrow$  mushroom
```

Aunque a primera vista pueda parecer funcional y correcto, el problema no es su corrección sino su incapacidad estructural para escalar:



- Con situaciones mixtas (enemigos en ambos lados, hazards y recta simultáneamente), el árbol de condicionales crece exponencialmente sin una solución limpia.
- El sistema fue diseñado para soportar bots con personalidades distintas, por lo que con reglas fijas habría que mantener un sistema completamente distinto por personalidad. En cambio, con un modelo supervisado, la personalidad emerge de la distribución del dataset de entrenamiento: el mismo pipeline genera modelos con estilos distintos cambiando solo los datos.

## 7.2.2 Estructura de dataset de entreno

| Columna          | Rango de valores                |
|------------------|---------------------------------|
| recta            | true/false                      |
| enemigos_delante | 0-5                             |
| enemigos_atras   | 0-5                             |
| platanos_delante | 0-5                             |
| conchas_atras    | 0-5                             |
| power_up_tirado  | {banana, shell, mushroom, star} |

Las columnas **platanos\_delante** y **conchas\_atras** son hazards específicos por tipo y dirección, no una detección genérica. Esta especificidad permite al modelo discriminar entre star (hazards en ambas direcciones) y banana (perseguidores detrás, camino despejado).

## 7.3 Dataset sintético y EDA

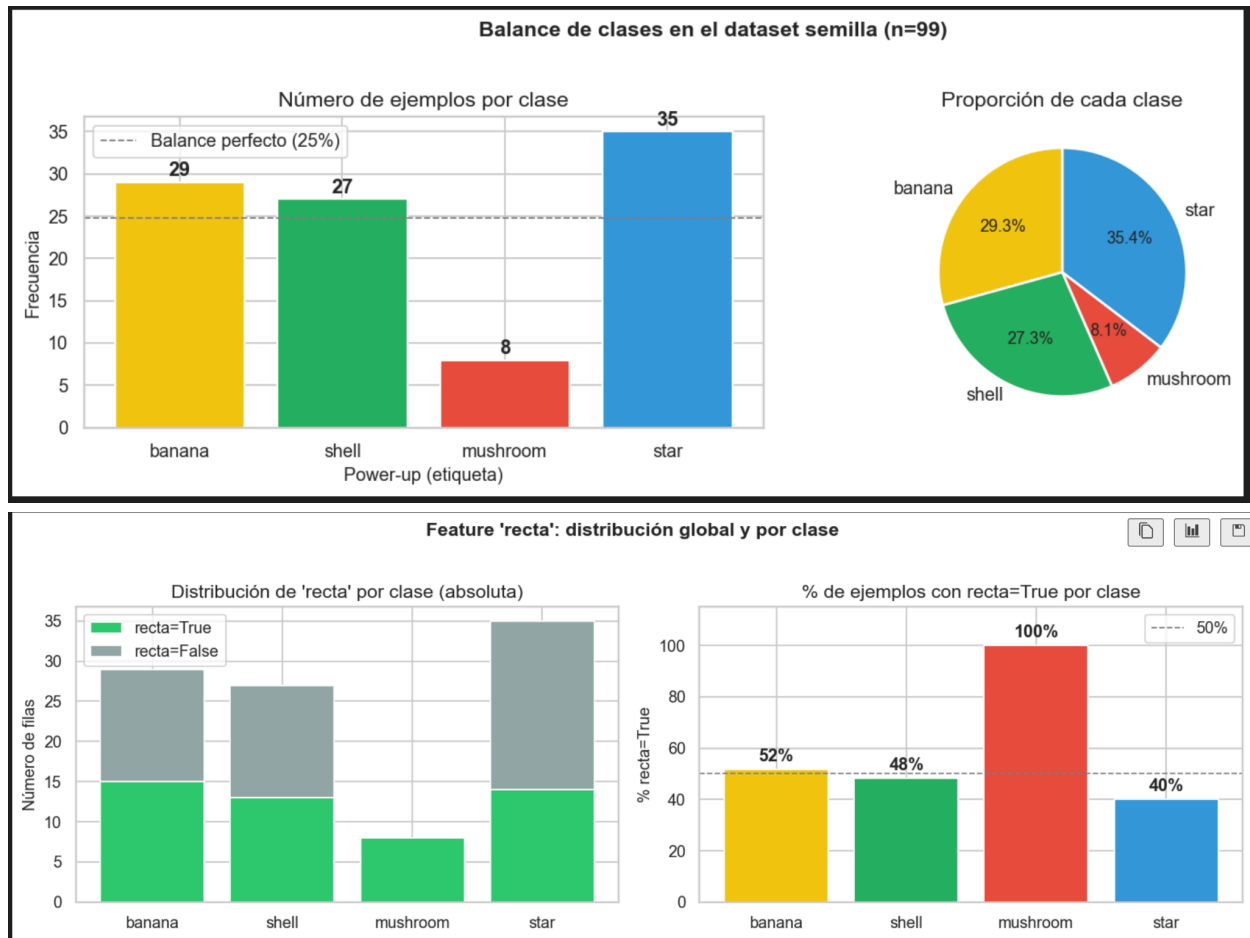
### 7.3.1 Creación del dataset semilla

Debido a nuestra condición de estudiantes y las limitaciones que eso conlleva, hemos recurrido al uso de datos sintéticos, ya que no disponemos del tiempo y los recursos para obtener datos reales de calidad. Reconocemos que no es lo recomendable, pero en una futura mejora y mantenimiento de este proyecto, la idea en mente es recopilar datos reales. Aún así hemos procurado sintetizar datos realistas.

Nuestro punto de partida es `powerups_sintetico.csv`: 99 filas creadas manualmente donde cada fila representa una situación concreta de carrera y el power-up correcto para ese contexto. La creación manual garantiza coherencia estratégica: cada ejemplo refleja una decisión razonada, no ruido estadístico.

El dataset semilla actúa como conocimiento experto codificado que, más tarde, el proceso de enriquecimiento amplificará de forma programática.

### 7.3.2 EDA del dataset semilla

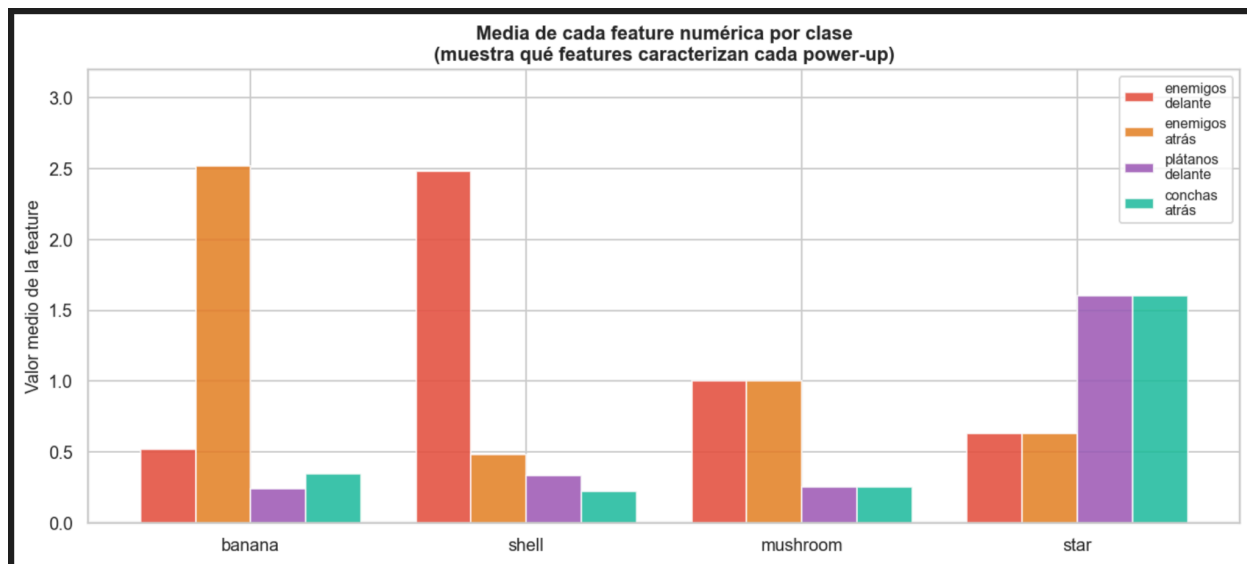
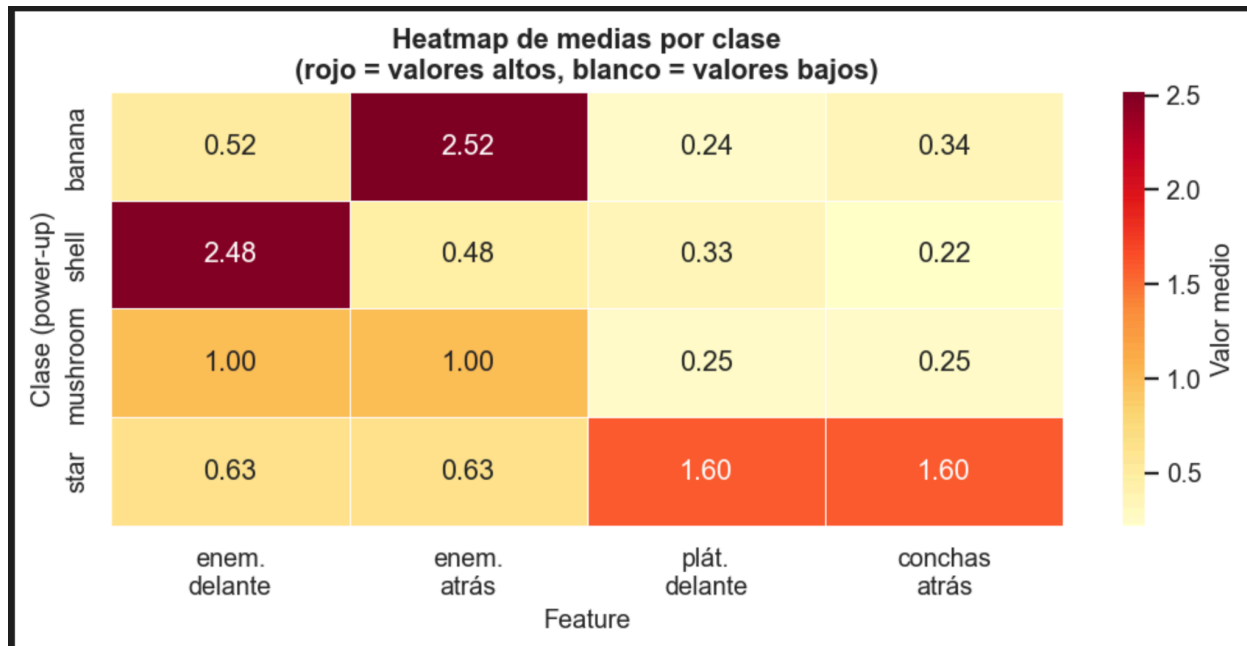


- **star** es la clase más representada (35 filas, 35.4%), seguida de 'banana' (29) y 'shell' (27). Estas tres tienen una distribución razonablemente equilibrada entre sí.
- **mushroom** está claramente infrarrepresentada: solo 8 filas (8.1%). Además todos sus ejemplos tienen **recta=True**.
- El ratio de desequilibrio es  $\sim 4.4\times$  (35 vs 8), lo que no es extremo pero sí suficientemente grande para que un modelo sin corrección tienda a ignorar mushroom.

#### Implicaciones para el entrenamiento:

Para aminorar este desequilibrio recurrimos a lo siguiente:

1. Usar **class\_weight='balanced'** en el **RandomForestClassifier**, que da más peso a los errores en clases minoritarias durante el entrenamiento.
2. En la fase de augmentation, los tres perfiles sintéticos (balanced, agresivo, pacífico) generan un mínimo de 500 filas de **mushroom**, corrigiendo el desequilibrio estructuralmente antes de que el modelo vea los datos.

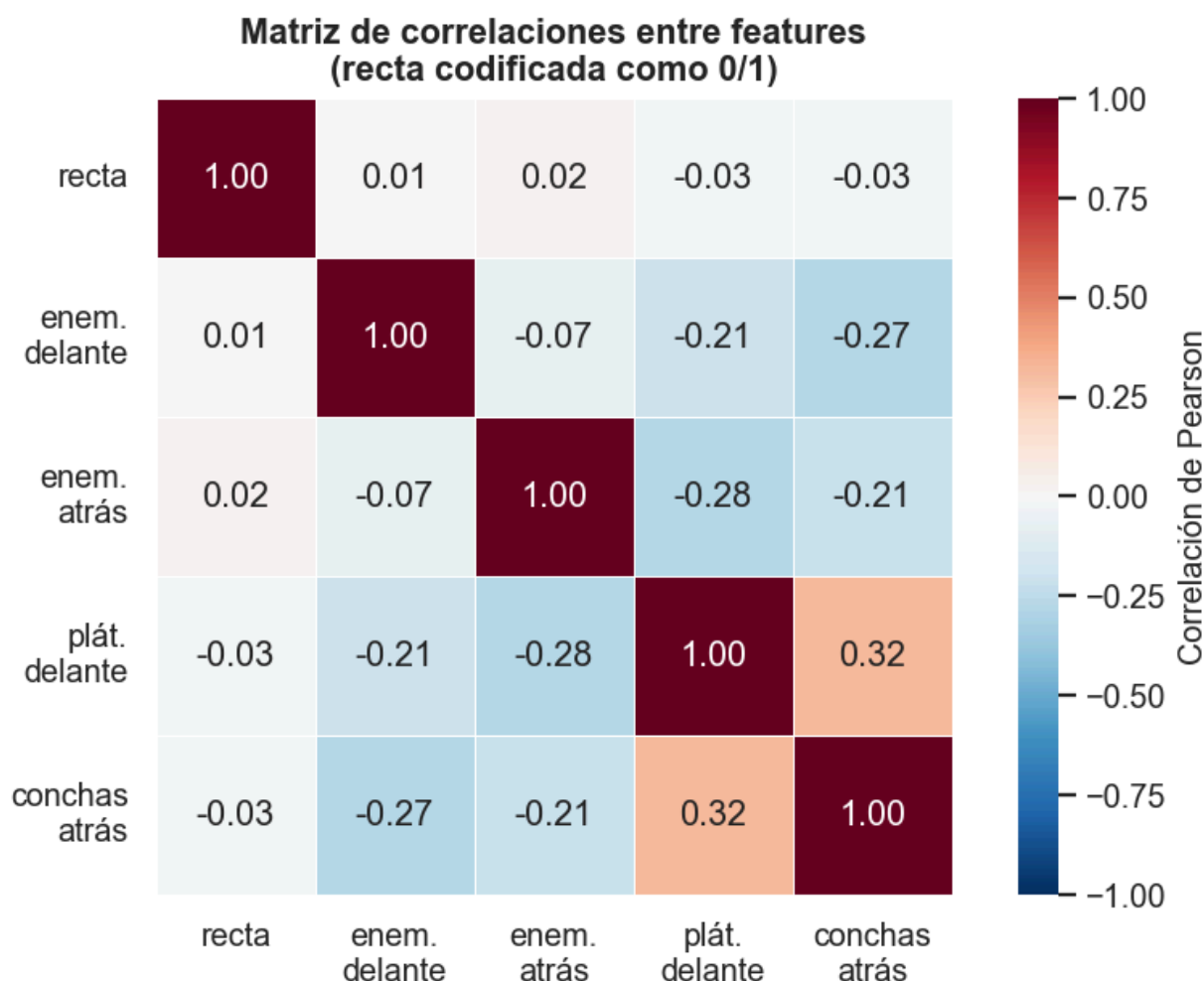


Estos gráficos ayudan a mostrar el *perfil típico* de cada clase:

| Clase    | Feature dominante                                   | Lectura estratégica                                                                                      |
|----------|-----------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| banana   | enemigos_atrás muy alto (~2.5)                      | Se usa para defenderse de quien persigue. enemigos_delante es bajo (~0.5), lo que la distingue de shell. |
| shell    | enemigos_delante muy alto (~2.5)                    | Se usa para atacar a quien va delante. enemigos_atrás es bajo (~0.5), simétrico a banana.                |
| mushroom | Todos los valores bajos + recta=True                | Se usa cuando no hay amenazas y estás en recta. Es el power-up de oportunidad limpia.                    |
| star     | platanos_delante y conchas_atrás altos (~1.6 ambos) | Se usa cuando hay hazards en ambas direcciones. Es la respuesta a amenazas de objeto.                    |

Esto demuestra que las cuatro clases tienen perfiles claramente distintos sin ningún par de clases que tenga exactamente el mismo perfil, lo que indica que el problema es linealmente separable y que un clasificador relativamente simple debería funcionar muy bien.

El único par con cierto solapamiento potencial es **banana** y **star**: ambas aparecen cuando **enemigos\_atras** es moderado-alto. La diferencia es que **star** también tiene **platanos\_delante** y **conchas\_atras** altos, mientras que **banana** no. La función de decisión en ese borde depende del nivel de hazards.



- **platanos\_delante** y **conchas\_atras** tienen correlación positiva alta: cuando hay plátanos en el camino también suele haber conchas atrás. Esto tiene sentido porque ambas situaciones co-ocurren en los ejemplos de **star** (la clase que requiere hazards en ambas direcciones).
- **enemigos\_delante** y **enemigos\_atras** tienen correlación negativa leve: cuando hay muchos enemigos delante tiende a haber pocos atrás y viceversa. Esto refleja la lógica de diseño del dataset: **shell** tiene enemigos delante pero no atrás, y **banana** tiene enemigos atrás pero no delante.
- **recta** tiene correlaciones bajas con todo: confirma que es una feature relativamente independiente, lo cual es correcto. La recta discrimina mushroom pero es irrelevante para las demás clases.

- No hay correlaciones extremas ( $>0.9$ ) que indiquen redundancia completa entre features. Las cinco features aportan información distinta al modelo.

## 7.4 Dataset sintético y Data Augmentation

### 7.4.1 Motivación y enfoque

Al igual que en el clasificador de comportamiento del jugador (que se aborda más adelante), y como ya hemos comentado antes, la falta de recursos para recolectar datos reales a escala nos obligó a recurrir a datos sintéticos. En este caso la situación es metodológicamente más sólida: el espacio de features es pequeño (5 dimensiones), el oráculo de etiquetado es explícito y verificable, y la semilla es la explicada en el apartado anterior.

La función **choose\_power\_up** actúa como oráculo determinístico. Cada fila sintética generada se valida contra este oráculo antes de ser aceptada: si la función no devuelve la etiqueta esperada, la fila se descarta y se genera otra. Esto garantiza coherencia total entre features y etiqueta en todo el dataset aumentado.

No se usó un LLM (a diferencia del clasificador de jugador) porque el espacio de features es lo suficientemente pequeño para formalizar las reglas de generación programáticamente.

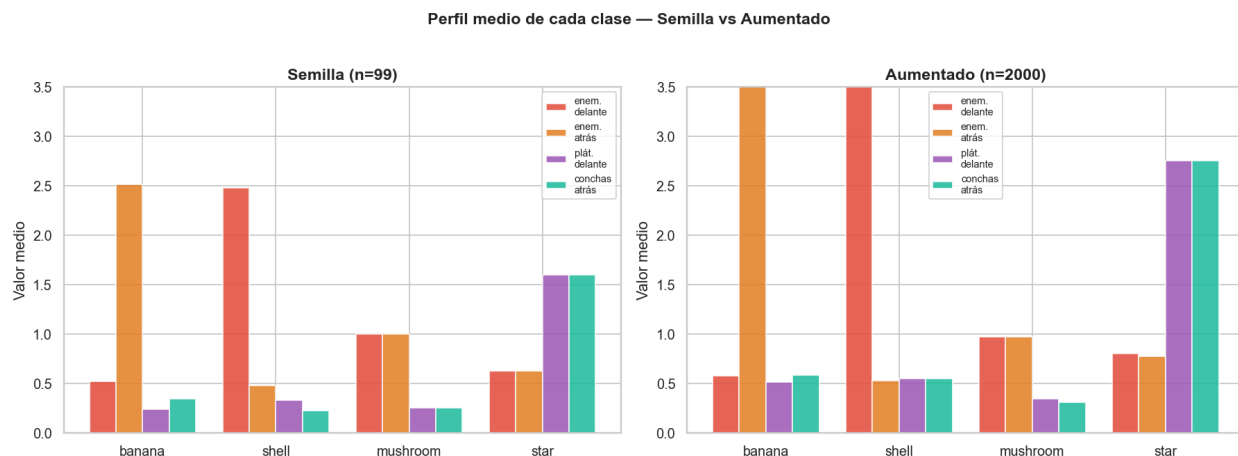
### 7.4.2 Mecanismo de generación

El script **Assets/Python/powerups\_augment\_dataset.py** es el encargado del data augmentation y generar datasets de tamaño arbitrario en tres pasos por fila:

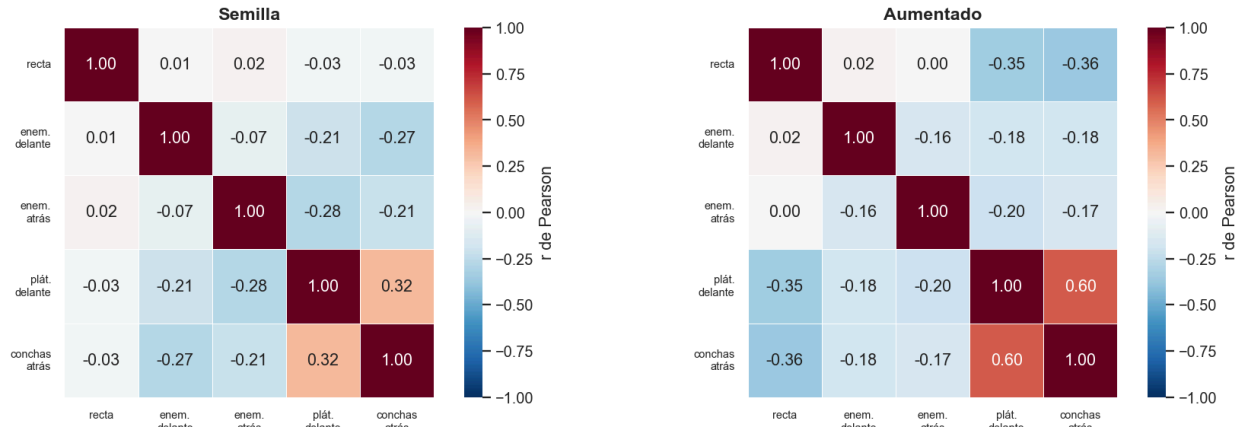
1. Selecciona una fila base de semilla de la clase objetivo.
2. Aplica jitter aleatorio a las features ( $\pm 1$  en enteros, 20% de probabilidad de cambio en recta) y sesga los valores hacia el contexto típico de la clase.
3. Valida con el oráculo. Si no pasa, reintenta hasta 200 veces.

### 7.4.3 Dataset de producción

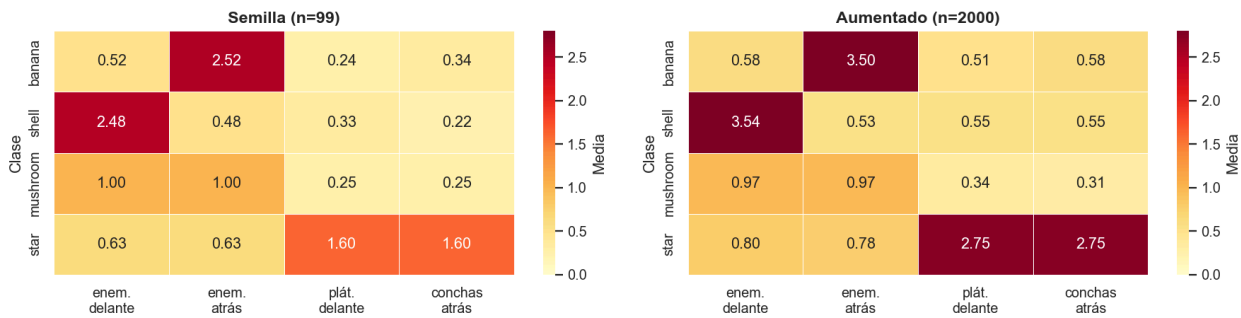
Tras obtener el dataset resultante, podemos volver a analizarlo estadísticamente.



Correlaciones entre features — Semilla vs Aumentado



Heatmap de medias por clase — misma escala de color



Comparando los dos heatmaps con la misma escala de color, se comprueba que los patrones de separación entre clases se han preservado:

- **shell** sigue teniendo el **enemigos\_delante** más alto de todas las clases.
- **banana** sigue teniendo el **enemigos\_atras** más alto.
- **star** sigue teniendo **platanos\_delante** y **conchas\_atras** altos.
- **mushroom** sigue teniendo los valores numéricos más bajos.

Lo que sí ha cambiado es, obviamente, la magnitud de los valores: las medias en el dataset aumentado son sistemáticamente más altas que en la semilla. El augmentation sesga cada fila hacia los valores típicos de su clase (p.ej., fuerza **enemigos\_atras** hacia valores altos en las filas de **banana**), lo que amplifica los contrastes entre clases. El modelo del dataset aumentado tendrá fronteras de decisión más claras que el entrenamiento directamente sobre la semilla.

El único inconveniente real que había era el desequilibrio de **mushroom**, pero gracias al data augmentation, ahora está resuelto. El dataset aumentado tiene exactamente 500 filas por clase. El ratio de desequilibrio pasa de 4.4x a 1.0x.

## 7.4.4 Limitaciones de los datos sintéticos

Entrenar con datos sintéticos introduce un riesgo de generalización: el modelo aprende los patrones definidos por el oráculo, no los patrones exhibidos por bots reales en partida. La brecha entre accuracy de test y comportamiento in-game refleja esta limitación.

## 7.5 Entrenamiento del Modelo

### 7.5.1 Uso de Random Forest

Se eligió Random Forest con una motivación importante: su estructura de árboles se serializa íntegramente a JSON y se reconstruye en C# sin ninguna dependencia de librerías ML en runtime. El juego puede distribuirse como build standalone con los modelos dentro del binario.

Además es esta motivación, existen otras razones para su uso:

- Maneja features mixtas (booleano + enteros) sin escalado previo.
- **class\_weight='balanced'** nativo para el desequilibrio de mushroom.
- **feature\_importances** interpretables que permiten validar que el modelo aprendió la lógica correcta.
- Inferencia < 1 ms por predicción.

No se realizó una comparación sistemática de múltiples modelos como en el clasificador de jugador porque en el momento de desarrollo no conocíamos otras alternativas a la restricción de serialización a C# nativo.

### 7.5.2 Hiperparámetros

| Hiperparámetro   | Valor      | Justificación                                                              |
|------------------|------------|----------------------------------------------------------------------------|
| n_estimators     | 300        | Reduce varianza; tamaño de modelo aceptable.                               |
| max_depth        | 10         | Evita overfitting con 2000 filas.                                          |
| min_samples_leaf | 2          | Regularización ligera contra hojas de una muestra.                         |
| max_features     | 'sqrt'     | $\sqrt{5} \approx 2$ features por split; aumenta diversidad entre árboles. |
| class_weight     | 'balanced' | Compensa el desequilibrio de mushroom.                                     |
| random_state     | 42         | Reproducibilidad total.                                                    |

La profundidad real de los árboles es: mínima 9, máxima 10, media 9.8. El límite max\_depth=10 es alcanzado por la mayoría de árboles, lo que indica que el dataset de 2000 filas con 5 features explota toda la capacidad disponible.

### 7.5.3 Pipeline de entrenamiento

El script **train\_powerups\_random\_forest.py** es el encargado del entrenamiento y realiza los siguientes pasos:

1. Carga el CSV aumentado.
2. Convierte la columna recta (true/false) a entero (1/0).
3. Aplica **train\_test\_split** estratificado por clase (test\_size=0.2, random\_state=42).
4. Entrena el **RandomForestClassifier** con los hiperparámetros anteriores.
5. Evalúa sobre el test set y reporta accuracy, classification report y matriz de confusión.
6. Guarda el modelo en formato joblib junto con metadatos.

## 7.6 Resultados de Evaluación

### 7.6.1 Métricas globales

| Métrica                          | Valor   |
|----------------------------------|---------|
| Accuracy sobre test set (n=400)  | 99.25%  |
| Precisión media (macro)          | 99.2%   |
| Recall medio (macro)             | 99.3%   |
| F1-score medio (macro)           | 99.2%   |
| Errores sobre el test set        | 3 / 400 |
| Profundidad media de los árboles | 9.8     |

### 7.6.2 Classification report por clase

| Clase    | Precisión | Recall | F1-score | Support |
|----------|-----------|--------|----------|---------|
| banana   | 0.990     | 0.990  | 0.990    | 100     |
| mushroom | 1.000     | 1.000  | 1.000    | 100     |
| shell    | 0.990     | 1.000  | 0.995    | 100     |
| star     | 0.990     | 0.980  | 0.985    | 100     |

El modelo alcanza un 99.25% de accuracy sobre el test set de 400 ejemplos, con solo 3 errores. Las métricas macro — precisión 99.2%, recall 99.3%, F1 99.2%. Confirman que este rendimiento es uniforme entre las cuatro clases y no está inflado por ninguna clase mayoritaria.

Los dos heatmaps del EDA ya anticiparon el único solapamiento real del problema: banana y star comparten contextos donde **enemigos\_atras** es moderado y los hazards son bajos pero



no nulos. Es el borde de decisión ambiguo del espacio de features, y es donde caen los 3 errores.

La profundidad media de 9.8 sobre un límite de 10 indica que el modelo está aprovechando toda la capacidad disponible. Con 5 features y un dataset de 2000 filas bien separadas, los árboles alcanzan el tope antes de quedarse sin información relevante que particionar. Esto es coherente con lo que el EDA mostraba: las cuatro clases son separables y el modelo tiene suficiente señal para aprender fronteras limpias.

Tenemos en cuenta el sesgo de estas métricas: el test set es sintético, generado por el mismo proceso de augmentation que el conjunto de entrenamiento. El 99.25% mide con qué precisión el modelo aprendió la distribución definida por el oráculo heurístico, no su generalización a decisiones reales de bots o jugadores humanos en partida.

### 7.6.3 Inferencia en el runtime (C#)

La lógica de inferencia está en **RandomForestPowerUpBrain.cs**. El bot evalúa el contexto de forma continua cada 0.25 segundos, independientemente de cuándo recogió el power-up. Cuando tiene puntos disponibles y la carrera está activa:

1. Construye el vector de 5 features del contexto actual.
2. Recorre los 300 árboles: en cada nodo compara `featureValues[featureIndices[nodo]]` contra `thresholds[nodo]` y navega al hijo izquierdo o derecho.
3. Acumula el voto de la hoja alcanzada en cada árbol.
4. Produce una lista ordenada por votos, no solo la clase ganadora.
5. Intenta usar el power-up más votado. Si no está disponible en inventario, intenta el siguiente en la lista.

El mecanismo de fallback por ranking garantiza que el bot use siempre el power-up contextualmente más adecuado entre los disponibles. La inferencia completa se ejecuta en menos de 1 ms (~3000 comparaciones por predicción).

## 7.8 Evaluación In-Game

### 7.8.1 Distribuciones de power-ups observada

| Power-Up | Usos | %     |
|----------|------|-------|
| mushroom | 47   | 43.5% |
| shell    | 33   | 30.6% |
| banana   | 20   | 18.5% |
| star     | 8    | 7.4%  |

La distribución refleja las características del circuito: predominio de tramos rectos (**mushroom** frecuente) y contacto directo con bots (**shell** relevante). El power-up **star** es poco frecuente porque los hazards activos simultáneos en ambas direcciones no ocurren con frecuencia en partidas normales.

## 8. Clasificador de comportamiento del jugador

El proyecto incluye un tercer modelo de aprendizaje supervisado, complementario al sistema de power-ups: un clasificador de comportamiento del jugador. Su objetivo es asignar a un jugador una etiqueta de estilo de juego al finalizar la carrera, basándose en los datos recolectados de su partida.

### 8.1 Arquetipos de Jugador

Tenemos definidas seis categorías de comportamiento basadas en métricas obtenidas dentro del juego.

| Atributo     | Valor                                                                                                                                                                           |
|--------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pro          | Tiempos de vuelta rápidos y consistentes. Finaliza en las primeras posiciones. Mínimas colisiones con paredes. Usa los objetos de forma estratégica. Rara vez recibe impactos.  |
| Aggressive   | Rápido pero arriesgado. Prioriza atacar a otros karts sobre la limpieza en carrera. Alto uso de objetos ofensivos, muchos impactos recibidos.                                   |
| Conservative | Tiempos de vuelta moderados. Evita el conflicto. Bajo uso de objetos ofensivos, casi nunca recibe impactos. Prioriza terminar la carrera sin incidentes.                        |
| Chaotic      | Altamente impredecible en todas las métricas. Alta varianza en tiempos de vuelta y posiciones. Uso de objetos aleatorio.                                                        |
| Clumsy       | Vueltas lentas. Posiciones consistentemente malas. Número muy alto de colisiones con paredes. Bajo uso de objetos. Recibe impactos frecuentemente por mala conciencia espacial. |
| Slow         | Simplemente lento. Posiciones bajas pero no torpe: pocas colisiones con paredes. Escaso uso de objetos.                                                                         |

Estas categorías fueron elegidas porque representan intenciones estratégicas distintas (Pro vs Conservative vs Aggressive), modos de fallo de habilidad diferenciados (Clumsy vs Slow) y una categoría comodín para comportamientos impredecibles (Chaotic). Son lo suficientemente amplias para ser aprendibles a partir de estadísticas agregadas de carrera, pero lo suficientemente específicas para ser accionables en la adaptación del juego.

## 8.2 Recolección de datos

### 8.2.1 PlayerLapDataRecorder

Desarrollamos el componente de Unity; **PlayerLapDataRecorder.cs**, que se adjunta automáticamente al kart del jugador en tiempo de ejecución. Este componente escucha eventos del juego — completar vueltas, usar objetos, recibir impactos de objetos y colisiones físicas — y registra las siguientes nueve métricas por vuelta:

| Columna                  | Descripción                                             |
|--------------------------|---------------------------------------------------------|
| lapTime_delta            | Segundos necesarios para completar la vuelta            |
| position_delta           | Posición en carrera al terminar la vuelta (1 = primero) |
| mapCollisions_total      | Colisiones con paredes y zonas fuera de pista           |
| shellsUsed_total         | Conchas lanzadas                                        |
| bananasUsed_total        | Plátanos lanzados                                       |
| mushroomsUsed_total      | Turbos de seta usados                                   |
| starsUsed_total          | Estrellas usadas                                        |
| bananaHitsReceived_total | Veces golpeado por plátano enemigo                      |
| shellHitsReceived_total  | Veces golpeado por concha enemiga                       |

El clasificador opera a nivel de carrera, no a nivel de vuelta. Una sola vuelta es una señal insuficiente: un jugador Pro puede tener una mala vuelta, y un jugador Chaotic puede tener una vuelta afortunada. Las tres vueltas de una carrera se agrupan en una sola fila antes de la clasificación:

- **lapTime\_delta** y **position\_delta** se convierten en la media de las tres vueltas.
- Todas las columnas de colisiones y objetos se convierten en el total de las tres vueltas.

Mediante esta agrupación conseguimos un perfil más estable y representativo del comportamiento del jugador durante esa carrera.

## 8.3 Datos Sintéticos

### 8.3.1 Limitaciones

La clasificación supervisada requiere datos de entrenamiento etiquetados — en este caso, carreras donde se obtengan datos reales del jugador para, más tarde, su clasificación.

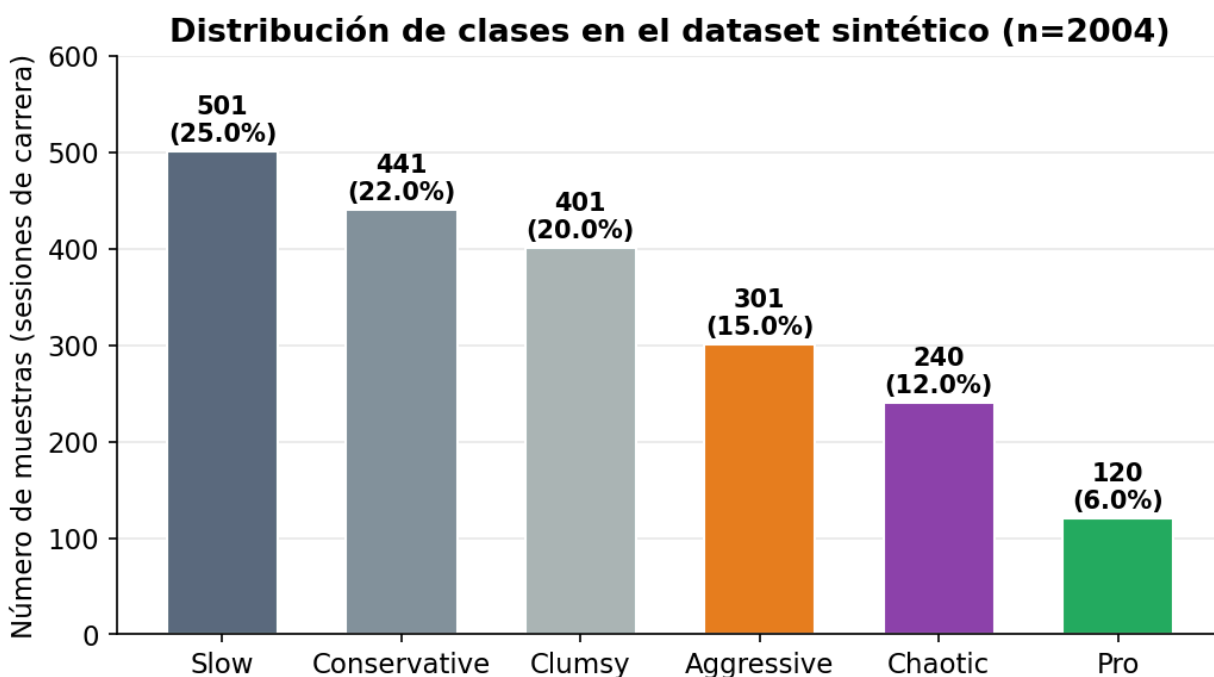
Sin embargo, al ser este un proyecto de fin de curso de estudiantes de máster, carecemos de tiempo y recursos para llevar a cabo una recolección real de datos suficiente para el entreno de nuestro modelo.

Conscientes de ello, recolectamos muestras reales mínimas y recurrimos a los datos sintéticos.

### 8.3.2 Diseño del Dataset Sintético

Usamos un LLM de gran escala (Claude Sonnet) para generar 2004 sesiones de carrera distribuidas entre los seis arquetipos, mediante un prompt estructurado que especificaba rangos numéricos exactos por arquetipo, restricciones de realismo (p. ej. el tiempo de vuelta debe correlacionar positivamente con la posición, más colisiones deben ralentizar la vuelta) y un desequilibrio de clases deliberado.

La **distribución de clases** se hizo deliberadamente desigual para reflejar poblaciones de jugadores realistas:



| Arquetipo    | Filas | Características                                                       |
|--------------|-------|-----------------------------------------------------------------------|
| Slow         | 25%   | Los jugadores casuales son la mayoría en cualquier juego              |
| Conservative | 22%   | El juego seguro es la estrategia más común entre jugadores no novatos |
| Clumsy       | 20%   | Las dificultades de control son comunes entre jugadores nuevos        |
| Aggressive   | 15%   | Un estilo de juego específico adquirido                               |
| Chaotic      | 12%   | Los jugadores erráticos existen pero no son la norma                  |
| Pro          | 6%    | Los jugadores de élite son raros por definición                       |

Un dataset balanceado (igual número de filas por clase) distorsionaría la realidad y produciría un modelo que trata todos los arquetipos como igualmente probables en inferencia. Esto haría que el clasificador sobre predica clases raras como Pro al aplicarse a una base de usuarios real dominada por jugadores casuales.

### 8.3.3 Restricciones de Realismo Aplicadas

Se aplicaron las siguientes restricciones para evitar que el LLM generase datos estadísticamente limpios pero poco realistas:

- **lapTime\_delta** debe correlacionar positivamente con *position\_delta* (los jugadores más rápidos terminan en mejores posiciones).
- **lapTime\_delta** debe correlacionar positivamente con *mapCollisions\_total* (más colisiones conlleva una vuelta más lenta).
- **starsUsed\_total** debe ser 0 en al menos el 95% de las filas (las estrellas son objetos muy poco frecuentes).
- El total de objetos usados por carrera no debe superar 12 (límite físico dado la frecuencia de recogida y la duración de la carrera).
- Aproximadamente el 10% de las filas por clase se permite solapar con rangos de clases adyacentes para simular que un jugador tiene una carrera inusualmente buena o mala.

### 8.3.4 Limitaciones de los Datos Sintéticos

Entrenar exclusivamente con datos sintéticos introduce un riesgo fundamental de generalización: el clasificador aprende los patrones definidos en el prompt, no necesariamente los patrones exhibidos por jugadores humanos reales. Los jugadores reales no se ajustarán perfectamente a los arquetipos. Las métricas de rendimiento del clasificador reportadas representan un límite superior que se degradará al aplicarse a datos de juego real.

### 8.3.5 Feature Engineering

Se calcularon tres características derivadas a partir de las nueve columnas originales antes del entrenamiento del modelo:

| Característica     | Fórmula                                                                                      | Justificación                                                                                                                                                                                          |
|--------------------|----------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| aggressiveness     | shellsUsed + bananasUsed                                                                     | Combina todas las acciones ofensivas con objetos en una única puntuación de intención ofensiva.                                                                                                        |
| hit_ratio          | $\frac{(\text{bananaHitsReceived} + \text{shellHitsReceived})}{(\text{aggressiveness} + 1)}$ | Mide con qué frecuencia un jugador es penalizado en relación con cuánto ataca. Valores altos indican mala posición o alta represalia. El +1 evita la división por cero.                                |
| powerup_efficiency | $\text{aggressiveness} / (\text{mapCollisions} + 1)$                                         | Captura si un jugador puede atacar mientras permanece en pista. Valores altos distinguen Aggressive de Clumsy: ambos pueden colisionar, pero solo Aggressive lo hace mientras usa objetos activamente. |

Estas características se motivaron al observar que las columnas originales no separan individualmente los seis arquetipos. En particular, Aggressive y Clumsy comparten recuentos elevados de colisiones; **powerup\_efficiency** es el principal discriminador entre ellos.

## 8.4 Entreno y comparación de modelos

### 8.4.1 División de los Datos

El dataset se dividió mediante muestreo estratificado para preservar las proporciones de clase en cada partición:

- **Conjunto de entrenamiento:** 70% (1.402 filas)
- **Conjunto de validación:** 15% (301 filas) — usado para la selección del modelo
- **Conjunto de prueba:** 15% (301 filas) — reservado hasta la evaluación final

La estratificación garantiza que la clase minoritaria (Pro, 120 filas) esté representada en las tres particiones.

### 8.4.2 Gestión del Desequilibrio de Clases

Todos los modelos se configuraron para tener en cuenta el desequilibrio de clases, ya que, sino, los modelos estarían incentivados a predecir la clase mayoritaria (Slow) y descuidar las clases minoritarias (Pro, Chaotic):

- Modelos con soporte nativo (`class_weight='balanced'`): Regresión Logística, Random Forest, SVM.
- Modelos que requieren pesos de muestra explícitos: Gradient Boosting, XGBoost, LightGBM — estos recibieron pesos por muestra calculados a partir de las frecuencias de clase mediante `compute_sample_weight('balanced')`.

### 8.4.3 Métrica de Evaluación

Se seleccionó **F1 Macro** como métrica principal para la selección del modelo. F1 Macro calcula el F1 de forma independiente para cada clase y los promedia sin ponderar por la frecuencia de clase. Esto trata una clasificación errónea de un jugador Pro (120 muestras de entrenamiento) como igualmente importante que una de un jugador Slow (501 muestras).

El Accuracy se registró pero no se usó para la selección. Con un desequilibrio de clases del 25% para la clase mayoritaria, un modelo que siempre prediga Slow alcanzaría una Accuracy del 25% sin aprender nada: el Accuracy es por tanto engañosa como métrica principal (cosa comentada en clase).

También se usa el F1 Ponderado. A diferencia del F1 Macro, pondera cada clase por su soporte, haciéndolo sensible al rendimiento en la clase mayoritaria. Se reportó por completitud pero no se usó para la selección como métrica prioritaria, dado que el objetivo es un rendimiento equitativo en todos los arquetipos.

### 8.4.4 Modelos Evaluados

Dado que nos enfrentamos a un problema de clasificación, se escogieron los siguientes modelos que conocemos gracias a proyectos anteriores:

| Modelo              | Descripción                                                          |
|---------------------|----------------------------------------------------------------------|
| Logistic Regression | Baseline interpretable; establece un límite inferior                 |
| Random Forest       | Sólido en datos tabulares; gestiona el desequilibrio de forma nativa |
| SVM (kernel RBF)    | Eficaz en escenarios de alto margen; incluido por completitud        |
| Gradient Boosting   | Boosting secuencial; sólido en datos estructurados                   |
| XGBoost             | Gradient boosting regularizado; estado del arte en tareas tabulares  |
| LightGBM            | Boosting hoja a hoja; más rápido que XGBoost, precisión competitiva  |
| MLP                 | Incluido para evaluar si una pequeña red neuronal aporta valor       |



#### 8.4.5 Resultados en el Conjunto de Validación

| Modelo              | Accuracy | F1 Macro | F1 Ponderado |
|---------------------|----------|----------|--------------|
| Random Forest       | 0.9635   | 0.9518   | 0.9625       |
| XGBoost             | 0.9601   | 0.9481   | 0.9598       |
| LightGBM            | 0.9601   | 0.9462   | 0.9601       |
| Gradient Boosting   | 0.9535   | 0.9419   | 0.9527       |
| SVM (RBF)           | 0.9468   | 0.9310   | 0.9451       |
| Logistic Regression | 0.9169   | 0.8871   | 0.9166       |
| MLP                 | 0.9203   | 0.8832   | 0.9180       |

Se puede observar que el modelo Random Forest tiene mayor F1 Macro en el conjunto de validación (0.9518), por lo que fue seleccionado como modelo de producción. Además, en nuestro contexto, los árboles de Random Forest pueden serializarse como estructura JSON y ejecutarse de forma nativa en C# dentro de Unity sin ninguna dependencia de librería ML en tiempo de ejecución.

#### 8.4.6 Validación Cruzada

Para verificar que los resultados no son un artefacto de la división entrenamiento/validación concreta, se realizó una validación cruzada estratificada de 5 pliegues sobre el conjunto combinado de entrenamiento y validación:

**F1 Macro: 0.9692 ± 0.0036**

Puntuaciones por pliegue: [0.9682, 0.9626, 0.9729, 0.9711, 0.9710]

La baja desviación estándar (0.0036) confirma que el rendimiento del modelo es estable entre distintas particiones de datos y no es sensible a la división aleatoria concreta.

## 9. Limitaciones y trabajo futuro

### 9.1 Limitaciones actuales

#### 9.1.1 En el bloque de IA

##### **KartAgent — generalización limitada a un único circuito.**

El agente fue entrenado exclusivamente sobre la pista de entrenamiento. No se ha validado su comportamiento en circuitos con geometría o curvas distintas, por lo que no es posible garantizar que el modelo generalice más allá del entorno de entrenamiento.

##### **KartAgent — no integrado en los bots de carrera.**

A pesar de haber completado el entrenamiento con éxito (recompensa acumulada convergente en 60–70), el modelo .onnx no se llegó a asignar a los bots de carrera en la escena final. Los bots siguen usando el controlador heurístico AIKartInput.

##### **PowerUpAgent (PPO) — rendimiento inferior al Random Forest.**

Se entrenó un agente PPO para la decisión de power-ups, pero su rendimiento en el contexto de este problema fue inferior al de los modelos Random Forest entrenados. El espacio de decisión reducido y la señal de recompensa discreta favorecen el enfoque supervisado, por lo que PPO fue descartado en favor del RF.

##### **Dataset de power-ups de tamaño reducido.**

El dataset semilla tiene 99 filas generadas sintéticamente a partir de decisiones de diseño manual, no de observaciones de juego real competitivo. Aunque fue aumentado a 2000 filas mediante data augmentation, las distribuciones reflejan los arquetipos definidos a priori y no el comportamiento real de jugadores humanos.

##### **Brecha sintético-real en el clasificador de jugador.**

El clasificador fue entrenado con datos sintéticos. La evaluación in-game (sección 9.12) muestra una accuracy media del **40.1%** (rango 20–72%), frente al **99.25%** obtenido en el test set. La brecha indica que la distribución de features en partida real no coincide con la distribución sintética de entrenamiento.

##### **Arquetipos definidos a priori.**

Las seis categorías del clasificador (Aggressive, Chaotic, Clumsy, Conservative, Pro, Slow) se definieron antes de la recogida de datos. El comportamiento real de los jugadores puede no ajustarse a estos clústeres. Una versión futura podría descubrir arquetipos mediante clustering no supervisado sobre datos reales en lugar de imponer categorías predefinidas.

#### 9.1.2 En el entorno de videojuego

- Solo un circuito principal funcional; los escenarios adicionales no llegaron a completarse.
- Sin sonido ni música.
- Sin efectos visuales avanzados en los power-ups.

## 9.2 Trabajo futuro

- **Integrar el KartAgent en los bots de carrera.** Asignar el modelo .onnx entrenado a los bots de la escena principal, reemplazando AIKartInput.
- **Generalizar el agente a múltiples circuitos** mediante curriculum learning, pasando progresivamente de segmentos simples a trazados más complejos.
- **Completar el entrenamiento por imitación** combinando demostraciones .demo grabadas por el jugador con PPO+GAIL.
- **Estudio A/B con los cuatro modelos Random Forest de power-ups** (base, agresivo, pacífico, ImitarPlayer) en partida real, midiendo posición media y tasa de acierto de power-ups.
- **Reentrenar el clasificador con datos reales.** Una vez disponibles suficientes partidas etiquetadas, mezclar datos sintéticos y reales para reducir la brecha de accuracy observada.
- **Aplicar clustering no supervisado** (k-means, DBSCAN) sobre datos de juego reales para validar o redefinir los arquetipos de comportamiento del jugador.
- Añadir sonido, música y efectos visuales de power-ups para completar la experiencia de juego.

## 10. Conclusiones

El desarrollo de este proyecto demostró que integrar múltiples técnicas de IA en un entorno de videojuego en tiempo real es tan un reto de ingeniería como de diseño. La lección más importante del entrenamiento PPO fue que el diseño de la función de recompensa es tan crítico como la elección del algoritmo: los primeros intentos con múltiples penalizaciones activas producían agentes que optimizan el ruido en lugar del objetivo real. Simplificar la señal fue lo que desbloquea la convergencia.

También aprendimos que no existe una única técnica de IA que resuelva todos los problemas de un juego. El aprendizaje por refuerzo resultó adecuado para la navegación compleja del circuito; el Random Forest, eficiente para clasificar comportamiento del jugador a partir de variables tabulares; y las reglas explícitas, insustituibles donde se necesita comportamiento predecible y ajustable por diseño.

Poner en práctica lo aprendido durante el curso y saber cuándo usar cada herramienta, y cómo hacerlas convivir, es la competencia central que este proyecto ha desarrollado.

## 11. Referencias

- Bakkes, S. C. J., Spronck, P., & van Lankveld, G. (2012). Player behavioural modelling for video games. *Entertainment Computing*, 3(3), 71–79. <https://doi.org/10.1016/j.entcom.2011.12.001>
- Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32. <https://doi.org/10.1023/A:1010933404324>
- Chen, M., Elmachtoub, A. N., & Lei, X. (2026). Matchmaking strategies for maximizing player engagement in video games. *Management Science*. <https://doi.org/10.1287/mnsc.2023.02957>
- Drachen, A., Canossa, A., & Yannakakis, G. N. (2009). Player modeling using self-organization in Tomb Raider: Underworld. In *Proceedings of the IEEE Symposium on Computational Intelligence and Games (CIG)*, 1–8. <https://doi.org/10.1109/CIG.2009.5286482>
- Etheredge, M., Lopes, R., & Bidarra, R. (2013). A generic method for classification of player behavior. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, 9(3), 2–8. <https://doi.org/10.1609/aiide.v9i3.12593>
- Iovino, M., Ögren, P., Colledanchise, M., & Smith, C. (2022). A survey of behavior trees in robotics and AI. *Robotics and Autonomous Systems*, 154, 104096. <https://doi.org/10.1016/j.robot.2022.104096>
- Juliani, A., Berges, V., Teng, E., Cohen, A., Harper, J., Elion, C., Goy, C., Gao, Y., Henry, H., Mattar, M., & Lange, D. (2020). Unity: A general platform for intelligent agents. *arXiv:1809.02627*. <https://arxiv.org/abs/1809.02627>
- Lara-Cabrera, R., Nogueira-Collazo, M., Cotta, C., & Fernández-Leiva, A. J. (2017). Adaptive user modelling in car racing games using behavioural and physiological data. *Cognitive Computation*, 9(6), 802–815. <https://doi.org/10.1007/s12559-017-9470-3>
- Machado, M. C., Fantini, E. P. C., & Chaimowicz, L. (2013). A methodology for player modeling based on machine learning. *arXiv:1312.3903*. <https://arxiv.org/abs/1312.3903>
- Schulman, J., Moritz, P., Levine, S., Jordan, M., & Abbeel, P. (2016). High-dimensional continuous control using generalized advantage estimation. In *Proceedings of the International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/1506.02438>
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms. *arXiv:1707.06347*. <https://arxiv.org/abs/1707.06347>
- Uludağlı, M. C., & Oğuz, O. S. (2023). Non-player character decision-making in computer games. *Artificial Intelligence Review*, 56(12), 14159–14191. <https://doi.org/10.1007/s10462-023-10491-7>

- Unity Technologies. (2024). *ML-Agents Toolkit Documentation*.  
<https://github.com/Unity-Technologies/ml-agents>
- Yannakakis, G. N., & Togelius, J. (2018). *Artificial Intelligence and Games*. Springer.  
<https://doi.org/10.1007/978-3-319-63519-4>